

EXPERIMENT 5: Create a Data Submission Form and Handle POST Requests

Aim:

To build an HTML form and handle its POST request in Express to collect and display submitted data.

Theory-

Forms are an essential way for users to interact with web applications — from submitting feedback and registering accounts to creating products or posting content. Understanding how forms work and how to handle **POST requests** is fundamental in server-side programming using Node.js and Express.

Form Submission in Web Applications

In traditional web development, an HTML form is used to collect data from users. The form element defines an action (the endpoint to send data to) and a method (GET or POST). While GET requests append data to the URL, POST requests send it securely within the body of the HTTP request.

Example:

```
<form action="/feedback" method="POST">
  <label>Name:</label>
  <input type="text" name="username">
  <label>Message:</label>
  <textarea name="message"></textarea>
  <button type="submit">Submit</button>
</form>
```

When the user submits the form, the browser sends the data to the server.

Handling POST Requests in Express

In Node.js with Express, we can handle form submissions using routes and middleware. Express automatically provides request parsing middleware for URL-encoded or JSON data.

```
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

Then, we define a POST route:

```
app.post('/feedback', (req, res) => {  
  const { username, message } = req.body;  
  res.send(`Thank you ${username}, your feedback was:  
  ${message}`);  
});
```

Here, `req.body` contains the data submitted through the form. The data can be validated, sanitized, and stored in a database for further use.

Dynamic Rendering after Submission

Server-side rendering using EJS or another templating engine allows displaying a confirmation page dynamically:

```
res.render('confirmation', { name: req.body.username, feedback:  
req.body.message });
```

The EJS file:

```
<h2>Feedback Confirmation</h2>  
<p>Thank you, <%= name %>! We received: "<%= feedback %>".</p>
```

Security and Validation

When handling form data, developers must ensure:

- **Input validation** (checking data type, format, and length).
- **Input sanitization** (removing harmful characters).
- **CSRF protection** (Cross-Site Request Forgery prevention).
- **Error handling** for missing or invalid data.

Importance of POST Handling

POST request handling forms the basis of almost every dynamic web application. Login forms, registration pages, comment systems, and dashboards all depend on it. Express simplifies this process, providing an elegant, modular way to manage input data securely.

Code-

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '', // keep empty unless set in XAMPP
  database: 'gamestore'
});

connection.connect((err) => {
  if (err) {
    console.error('✗ Database connection failed:', err.message);
  } else {
    console.log('☑ Connected to MySQL database');
  }
});

module.exports = connection;
```

Conclusion

Data submission and POST handling are integral to interactive applications. Express simplifies these interactions, allowing developers to process and display user data dynamically. Proper validation and secure handling ensure both functionality and safety in real-world web systems.

EXPERIMENT 6: Serve Dynamic Content Using Server-Side Rendering

Aim:

To dynamically render pages using EJS templates populated with server-side data.

Theory

In modern web development, websites are increasingly data-driven and dynamic. This means content displayed to the user changes depending on data, user actions, or context. One of the most efficient ways to achieve this in backend development is through **Server-Side Rendering (SSR)**. SSR is a technique where the server generates HTML content dynamically before sending it to the client's browser. In the Node.js ecosystem, this is often implemented using a templating engine like **EJS (Embedded JavaScript)**.

Concept of Server-Side Rendering

Server-Side Rendering works by combining templates with data on the server to produce complete HTML pages. When a client sends a request (for example, to `/products`), the server fetches data — either from a database or a local source — and injects it into a view template. The resulting HTML is then sent to the browser. This process differs from **Client-Side Rendering (CSR)**, where the browser downloads JavaScript files and renders content dynamically after loading. SSR provides faster initial load times and is better for SEO since search engines can crawl the rendered HTML.

EJS – Embedded JavaScript Templating

EJS is a lightweight templating language that lets you embed JavaScript logic directly within HTML. It supports features like looping, conditional statements, and partials. To set up EJS with Express:

```
app.set('view engine', 'ejs');
```

A simple route that renders a list of services:

```
app.get('/services', (req, res) => {  
  
  const services = [  
  
    { name: 'Web Design', price: 500 },  
  
    { name: 'Mobile Development', price: 800 },  

```

```

    { name: 'SEO Optimization', price: 300 }
  ];

  res.render('services', { services });
});

```

The EJS file (`services.ejs`):

```

<h2>Our Services</h2>

<ul>

  <% services.forEach(service => { %>

    <li><%= service.name %> - $<%= service.price %></li>

  <% }); %>

</ul>

```

This will render an HTML list dynamically populated with data.

Advantages of SSR

1. **Improved SEO:** Since content is rendered before reaching the browser, search engines can index pages easily.
2. **Faster first paint:** The user sees meaningful content quicker compared to client-rendered pages.
3. **Reduced JavaScript dependency:** The server handles most rendering logic.
4. **Better performance on low-end devices:** Less computation occurs on the client side.

Use Cases

- E-commerce sites displaying dynamic product listings.

- Blogs fetching posts from a database.
- Dashboards showing live updates.

Code-

```
<!DOCTYPE html>
<html lang="en">
  <%- include('../partials/head') %>
  <body>
    <%- include('../partials/header') %>

    <section class="hero">
      <div class="hero-content">
        <h1>IGNITE YOUR PASSION FOR THE ROAD</h1>
        <p>Explore the world of motorcycles – speed, design, and performance that
define true riders.</p>
        <a href="/bikes" class="btn">Explore Bikes</a>
      </div>
    </section>

    <section class="welcome">
      <h2>Welcome to Revved Riders</h2>
      <p>At Revved Riders, we celebrate the art of motorcycling. From powerful
superbikes to elegant cruisers,
      our platform connects bike enthusiasts and showcases stunning rides from
around the globe.</p>
    </section>

    <section class="featured">
      <h2>Upcoming Bikes</h2>
      <div class="bike-grid">
        <!-- Upcoming Bike 1 -->
        <div class="bike-card">
          
          <div class="coming-soon">Coming Soon</div>
          <h3>Yamaha R9</h3>
          <p>Expected Price: ₹12,00,000</p>
          <p>Launch: Jan 2026</p>
        </div>
      </div>
    </section>
```

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

</div>

<!-- Upcoming Bike 2 -->
<div class="bike-card">
```

```

    
    <div class="coming-soon">Coming Soon</div>
    <h3>Royal Enfield Shotgun 650</h3>
    <p>Expected Price: ₹3,50,000</p>
    <p>Launch: Dec 2025</p>
</div>

<!-- Upcoming Bike 3 -->
<div class="bike-card">
    
    <div class="coming-soon">Coming Soon</div>
    <h3>KTM Adventure 490</h3>
    <p>Expected Price: ₹4,20,000</p>
    <p>Launch: Feb 2026</p>
</div>

<!-- Upcoming Bike 4 -->
<div class="bike-card">
    
    <div class="coming-soon">Coming Soon</div>
    <h3>Honda CBR500R</h3>
    <p>Expected Price: ₹5,00,000</p>
    <p>Launch: Mar 2026</p>
</div>

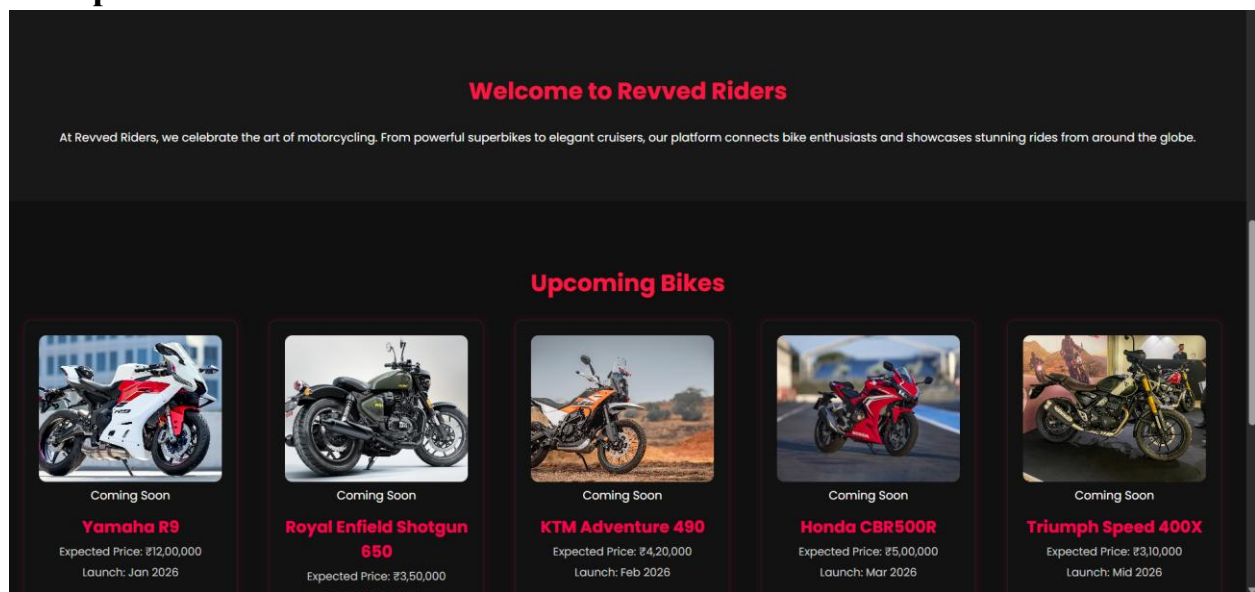
<!-- Upcoming Bike 5 -->
<div class="bike-card">
    
    <div class="coming-soon">Coming Soon</div>
    <h3>Triumph Speed 400X</h3>
    <p>Expected Price: ₹3,10,000</p>
    <p>Launch: Mid 2026</p>
</div>

<!-- Upcoming Bike 6 -->
<div class="bike-card">
    
    <div class="coming-soon">Coming Soon</div>
    <h3>BMW G310 Adventure</h3>
    <p>Expected Price: ₹4,50,000</p>
    <p>Launch: Apr 2026</p>
</div>
</div>
</section>

<%- include('../partials/footer') %>
</body>
</html>

```


Output-



Conclusion

Server-Side Rendering using EJS provides an efficient method for creating fast, SEO-friendly, and dynamic websites. It bridges the gap between traditional static websites and modern interactive applications, offering the benefits of speed and maintainability.

7. Create Dynamic Detail Pages Using URL Parameters

Aim:

To generate dynamic pages that display item details using Express route parameters and EJS rendering.

Theory

In dynamic web applications, users often need to view details about specific items — like a product, article, or user profile. Rather than creating separate pages for each item, **URL parameters** allow developers to build a single, flexible route that dynamically generates content based on the provided parameter. In Express.js, this is achieved using **route parameters** (denoted by a colon `:` in the URL).

Understanding Route Parameters

A route parameter is a variable part of the URL that can capture values passed by the client. For instance, `/product/:id` defines a route pattern where `:id` is a placeholder for a dynamic value (e.g., `/product/2`). When a request is made, Express captures this value and makes it available under `req.params`.

Example:

```
app.get('/product/:id', (req, res) => {  
  
  const products = [  
  
    { id: 1, name: 'Gaming Laptop', price: 90000 },  
  
    { id: 2, name: 'Wireless Mouse', price: 1500 },  
  
  ];  
  
  const product = products.find(p => p.id == req.params.id);  
  
  res.render('productDetail', { product });  
  
});
```

EJS template:

```
<h2><%= product.name %></h2>

<p>Price: ₹<%= product.price %></p>
```

When the user navigates to `/product/1`, it will display details of the first product. This approach eliminates redundancy and enables a scalable application structure.

Chaining Multiple Parameters

Express supports multiple route parameters:

```
app.get('/category/:catId/product/:prodId', (req, res) => {

  res.send(`Category:      ${req.params.catId},      Product:
${req.params.prodId}`);

});
```

This feature is especially useful in e-commerce or content management systems.

Benefits

- **Scalability:** One template serves hundreds of pages dynamically.
- **Code reusability:** Reduces repetition of similar routes.
- **Improved UX:** Allows meaningful, readable URLs for users.
- **SEO optimization:** Search engines prefer structured URLs with identifiers or names.

Code: -

```
<%- include('../partials/head') %>
<%- include('../partials/header') %>

<section class="products-section">
  <h2>Our Products</h2>
  <div class="product-grid">
    <div class="product-card">
      
      <h3>Red Rooster Performance Exhaust</h3>
      <p>Performance Exhaust systems for motorcycles which are designed to
increase power and torque, reduce weight, and provide a distinctive sound</p>
      <p class="price">₹20,000</p>
      <form action="/add-to-cart" method="POST">
```

```

        <input type="hidden" name="name" value="Red Rooster Performance Exhaust">
        <input type="hidden" name="price" value="20000">
        <input type="hidden" name="image" value="/image/rrp.jpg">
        <button type="submit" class="btn-buy">🛒 Buy Now</button>
    </form>
</div>

<div class="product-card">
    
    <h3>Riding Sunglasses</h3>
    <p>Stylish UV-protected sunglasses for bikers.</p>
    <p class="price">₹1,200</p>
    <form action="/add-to-cart" method="POST">
        <input type="hidden" name="name" value="Riding Sunglasses">
        <input type="hidden" name="price" value="1200">
        <input type="hidden" name="image" value="/image/sunglasses.jpg">
        <button type="submit" class="btn-buy">🛒 Buy Now</button>
    </form>
</div>

<div class="product-card">
    
    <h3>Full-Face Helmet</h3>
    <p>IS-certified helmet for safe rides.</p>
    <p class="price">₹4,500</p>
    <form action="/add-to-cart" method="POST">
        <input type="hidden" name="name" value="Full-Face Helmet">
        <input type="hidden" name="price" value="4500">
        <input type="hidden" name="image" value="/image/helmet.jpg">
        <button type="submit" class="btn-buy">🛒 Buy Now</button>
    </form>
</div>
</div>
</section>

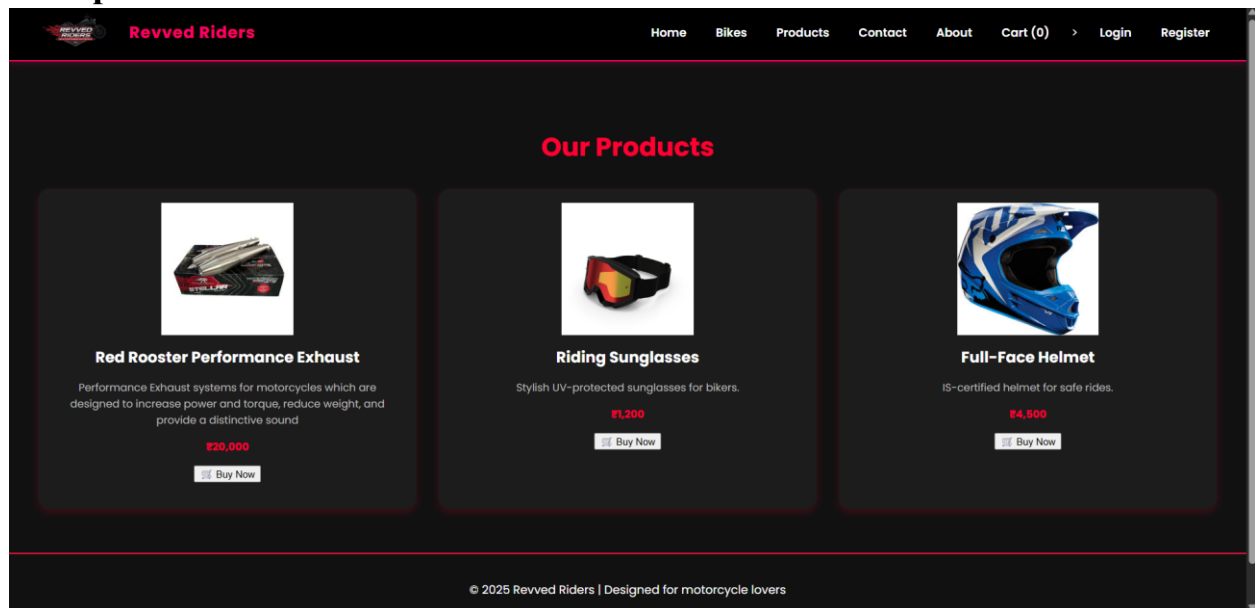
<%- include('../partials/footer') %>
<script>
document.querySelectorAll('.buy-btn').forEach(button => {
    button.addEventListener('click', async e => {
        e.preventDefault();
        const form = e.target.closest('form');
        const formData = new FormData(form);

        await fetch('/add-to-cart', {
            method: 'POST',
            body: formData
        });

        alert('Added to cart!');
    });
});
</script>

```

Output-



Conclusion

Using route parameters enables the creation of flexible, scalable, and data-driven web applications. Developers can dynamically render specific pages without duplicating code, improving both maintainability and performance.

8. Build a Reusable Layout Using EJS Partials

Aim:

To maintain UI consistency by creating reusable EJS partials for common components like header, footer, and navbar.

Theory

Consistency is vital in web design. Every page should share common elements like headers, footers, and navigation menus. Writing these components repeatedly across pages is inefficient. To solve this, templating engines like EJS introduce **partials** — reusable HTML snippets included in multiple templates.

Concept of Partials

Partials are fragments of HTML and EJS that can be embedded into other templates. For instance, a navigation bar can be written once as `navbar.ejs` and then included in every page using:

```
<%- include('partials/navbar') %>
```

This not only ensures uniformity across all pages but also reduces maintenance time. A change in the partial file automatically reflects across all pages that include it.

Example folder structure:

```
views/  
├─ partials/  
│   ├─ header.ejs  
│   └─ footer.ejs  
├─ home.ejs  
└─ about.ejs
```

header.ejs

```
<header class="bg-dark text-white p-3">
```

```
<h1>My Website</h1>

<nav><a href="/">Home</a> | <a href="/about">About</a></nav>

</header>
```

home.ejs

```
<%- include('partials/header') %>

<main>

  <h2>Welcome to the homepage!</h2>

</main>

<%- include('partials/footer') %>
```

Advantages

1. **Consistency:** All pages share the same UI components.
2. **Maintainability:** Update one file to modify all pages.
3. **Reusability:** Components can be reused in multiple projects.
4. **Reduced redundancy:** Less repetitive HTML code.

Conclusion

Using EJS partials creates clean, maintainable, and scalable UI architecture. It ensures a professional look, reduces development time, and simplifies large multi-page application management.

9. Connect Node.js to MySQL and Perform Basic CRUD

Aim:

To integrate Node.js with MySQL and perform CRUD (Create, Read, Update, Delete) operations.

Theory

For web applications to be practical, they must store and retrieve data persistently. While in-memory data disappears after server restart, databases like **MySQL** provide structured and durable data storage. Node.js interacts with MySQL using the `mysql2` or `sequelize` package, allowing the execution of **CRUD operations** (Create, Read, Update, Delete).

Connecting Node.js and MySQL

To connect:

```
const mysql = require('mysql2');

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'shopdb'
});

db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
```

CRUD Operations

1. Create (INSERT):

```
db.query('INSERT INTO products (name, price) VALUES (?, ?)',  
['Mouse', 500]);
```

2. Read (SELECT):

```
db.query('SELECT * FROM products', (err, results) =>  
console.log(results));
```

3. Update:

```
db.query('UPDATE products SET price=? WHERE id=?', [600, 1]);
```

4. Delete:

```
db.query('DELETE FROM products WHERE id=?', [1]);
```

Advantages

- MySQL provides relational data management using tables, primary keys, and relationships.
- Node.js allows asynchronous operations, improving performance for concurrent queries.
- CRUD operations form the backbone of all dynamic data applications.

Code-

```
// server.js
const express = require("express");
const session = require("express-session");
const path = require("path");
const mysql = require("mysql2");

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.static(path.join(__dirname, "public")));
app.use(express.urlencoded({ extended: true }));

app.use(
  session({
    secret: "revvedRiderSecretKey",
    resave: false,
    saveUninitialized: true,
    cookie: { maxAge: 1000 * 60 * 60 * 2 },
  })
);

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "revved_riders",
});

db.connect((err) => {
  if (err) console.error("MySQL connection failed:", err);
  else console.log("Connected to MySQL (revved)");
});

app.use((req, res, next) => {
  res.locals.user = req.session.user || null;
  res.locals.cart = req.session.cart || [];
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0;
  next();
});

function requireLogin(req, res, next) {
  if (!req.session.user) return res.redirect("/login");
  next();
}
```

```

app.use((req, res, next) => {
  res.locals.user = req.session.user;
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0;
  next();
});

app.get("/login", (req, res) => {
  if (req.session.user) return res.redirect("/");
  res.render("pages/login", { title: "Login", error: null });
});

app.post("/login", (req, res) => {
  const { email, password } = req.body;

  db.query(
    "SELECT * FROM users WHERE email = ? AND password = ?",
    [email, password],
    (err, results) => {
      if (err) {
        console.error("Login error:", err);
        return res.render("pages/login", { title: "Login", error: "Database error" });
      }
      if (results.length > 0) {
        const user = results[0];
        req.session.user = { name: user.name, email: user.email };
        if (!req.session.cart) req.session.cart = [];
        console.log("Login success:", user.email);
        return res.redirect("/");
      }
      res.render("pages/login", { title: "Login", error: "Invalid credentials" });
    }
  );
});

app.get("/register", (req, res) => {
  if (req.session.user) return res.redirect("/");
  res.render("pages/register", { title: "Register", error: null });
});

app.post("/register", (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password) {
    return res.render("pages/register", { title: "Register", error: "Fill all fields" });
  }

  db.query("SELECT * FROM users WHERE email = ?", [email], (err, results) => {
    if (err) {

```

```

        console.error("Register error:", err);
        return res.render("pages/register", { title: "Register", error: "Database
error" });
    }

    if (results.length > 0) {
        return res.render("pages/register", { title: "Register", error: "User
already exists" });
    }

    db.query(
        "INSERT INTO users (name, email, password) VALUES (?, ?, ?)",
        [name, email, password],
        (err) => {
            if (err) {
                console.error("Insert error:", err);
                return res.render("pages/register", { title: "Register", error:
"Database error" });
            }
            console.log("Registered new user:", email);
            res.redirect("/login");
        }
    );
});
});

app.get("/logout", (req, res) => {
    req.session.destroy((err) => {
        if (err) console.error("Session destroy error:", err);
        res.redirect("/login");
    });
});

app.get("/", (req, res) => res.render("pages/index", { title: "Home" }));
app.get("/bikes", (req, res) => res.render("pages/bikes", { title: "Bikes" }));
app.get("/products", (req, res) => res.render("pages/products", { title:
"Products" }));
app.get("/contact", (req, res) => res.render("pages/contact", { title: "Contact"
}));
app.get("/about", (req, res) => res.render("pages/about", { title: "About" }));

app.get("/cart", requireLogin, (req, res) => {
    const cart = req.session.cart || [];
    res.render("pages/cart", { title: "Your Cart", cart, message: null });
});

app.post("/add-to-cart", requireLogin, (req, res) => {
    const { name, price, image } = req.body;
    if (!req.session.cart) req.session.cart = [];
    req.session.cart.push({ name, price: parseFloat(price), image });
});

```

```
    console.log("Added to cart:", name);
    res.redirect("/products");
  });

app.post("/remove-item", requireLogin, (req, res) => {
  const { index } = req.body;
  if (req.session.cart && req.session.cart.length > index) {
    req.session.cart.splice(index, 1);
  }
  res.redirect("/cart");
});

app.post("/checkout", requireLogin, (req, res) => {
  req.session.cart = [];
  res.render("pages/cart", {
    title: "Your Cart",
    cart: [],
    message: "Checkout successful! Thank you.",
  });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
```

Output-

Register

Name

Email

Password

Register

Already have an account? [Login here](#)

Conclusion

Integrating Node.js with MySQL allows efficient data handling and persistence. CRUD operations enable complete database interactivity, making the web application functional and real-world ready.

10. Develop RESTful APIs for Data Access and Manipulation

Aim:

To build and test RESTful APIs for performing CRUD operations using [Express.js](#).

Theory

A **RESTful API (Representational State Transfer)** is a standardized way for systems to communicate over HTTP. In a REST API, each resource (like users or products) is identified by a unique URL, and different HTTP methods define the type of operation.

REST Architecture

REST uses:

- **GET** → Read data
- **POST** → Create data
- **PUT/PATCH** → Update data
- **DELETE** → Remove data

Example Express routes:

```
app.get('/api/products', (req, res) => res.json(products));

app.post('/api/products', (req, res) => {
  products.push(req.body); res.json({ message: 'Added' }); });

app.put('/api/products/:id', (req, res) => res.json({ message:
  'Updated' }));

app.delete('/api/products/:id', (req, res) => res.json({
  message: 'Deleted' }));
```

Testing with Postman

Postman allows developers to test API endpoints easily by sending requests and verifying JSON responses, ensuring reliability before integration with front-end apps.

Best Practices

- Use meaningful routes (`/api/users/:id` instead of `/getUserData`).
- Return consistent JSON responses with status codes.
- Implement error handling and input validation.
- Use middleware for logging and authentication.

Advantages

- Platform independence — APIs work across mobile, web, and desktop.
- Scalable and modular architecture.
- Facilitates separation between frontend and backend.

Code-

```
<!DOCTYPE html>
<html lang="en">
  <%- include('../partials/head') %>
  <body>
    <%- include('../partials/header') %>

    <section class="auth-section">
      <div class="auth-container">
        <h1>Register</h1>

        <% if (typeof error !== 'undefined' && error) { %>
          <p class="error"><%= error %></p>
        <% } %>

        <form action="/register" method="POST">
          <div class="form-group">
            <label>Name</label>
            <input type="text" name="name" placeholder="Enter your name"
required>
          </div>

          <div class="form-group">
            <label>Email</label>
            <input type="email" name="email" placeholder="Enter your email"
required>
          </div>

          <div class="form-group">
```



```
        <label>Password</label>
        <input type="password" name="password" placeholder="Create a
password" required>
    </div>

    <button type="submit" class="btn">Register</button>
</form>

    <p class="switch-text">Already have an account? <a href="/login">Login
here</a></p>
</div>
</section>

<%- include('../partials/footer') %>
</body>
</html>
```

Output-

Register

Name

Email

Password

Register

Already have an account? [Login here](#)

The image shows a terminal window and a web browser displaying phpMyAdmin. The terminal window shows the command `node server.js` being executed, with the output indicating the server is running at `http://localhost:3000` and connected to MySQL (revved). The phpMyAdmin interface shows the 'revved_riders' database selected, with the 'users' table chosen. The table structure is displayed, showing columns: id, name, email, and password. The table contains 7 rows of data.

Terminal output:

```
PS C:\Users\abhid\Desktop\Wt> node server.js
Server running at http://localhost:3000
Connected to MySQL (revved)
```

phpMyAdmin interface details:

- Server: 127.0.0.1 » Database: revved_riders » Table: users
- Showing rows 0 - 6 (7 total, Query took 0.0010 seconds.)
- SQL query: `SELECT * FROM `users``
- Number of rows: 25 (dropdown)
- Filter rows: Search this table
- Sort by key: None

	id	name	email	password
☐	1	Snehal	snehal036@gmail.com	1234
☐	2	Abhi	hehe@gmail.com	12345
☐	3	Shweta	hello@gmail.com	hiii3
☐	4	sujit	sujit@gmail.com	sujii
☐	5	Abhii	hello1@gmail.com	abhi03
☐	6	abhi	hehe3@gmail.com	12345
☐	7	Abhishek	abhidhautre@gmail.com	123456

Conclusion

RESTful APIs are the backbone of modern web services. Using Express.js, developers can build fast, secure, and scalable APIs to handle CRUD operations efficiently. They enable seamless data exchange across applications, forming the foundation of every full-stack ecosystem.